

КОНЦЕПТУАЛЬНЫЙ ДОКУМЕНТ · ВЕРСИЯ 2.0



CherryGit

Система диагностики процессов разработки программного обеспечения на основе анализа данных Git-репозитория

ВИДЕНИЕ СИСТЕМЫ И ПРЕДМЕТНОЙ ОБЛАСТИ

АВТОР

Яцук Сергей Николаевич

ДАТА

Май 2026 г.

СТАТУС

Концепция продукта · MVP в рамках ВКР

БАЗОВЫЕ МОДЕЛИ

DORA (5 метрик) · SPACE Framework

Содержание

- 1. Резюме концепции**
- 2. Цель системы и ожидаемый результат**
 - 2.1. Стратегическая цель
 - 2.2. Операционные цели
- 3. Задачи системы**
 - 3.1. Задачи уровня поставки ПО
 - 3.2. Задачи уровня код-ревью
 - 3.3. Задачи диагностики команды
 - 3.4. Задачи защиты прав пользователей
- 4. Целевая аудитория**
 - 4.1. Разработчик
 - 4.2. Тимлид команды
 - 4.3. Руководитель отдела разработки
 - 4.4. Организация в целом
- 5. Теоретическая основа: DORA и SPACE**
 - 5.1. Модель DORA (текущая редакция, 5 метрик)
 - 5.2. Модель SPACE
 - 5.3. Закон Гудхарта как методологический ограничитель
- 6. Принципы системы метрик**
- 7. Метрики системы**
 - 7.1. Метрики потока поставки (DORA)
 - 7.2. Метрики процесса код-ревью
 - 7.3. Метрики устойчивости команды
 - 7.4. Маппинг метрик на грани SPACE
 - 7.5. Намеренно исключённые метрики
- 8. Ролевая модель видимости данных**
- 9. Связь метрик с целями системы**
- 10. Граница MVP в рамках ВКР**
- 11. Заключение**

1. Резюме концепции

CherryGit — информационная система диагностики процессов разработки программного обеспечения, ориентированная на команды, использующие систему контроля версий Git и платформу GitLab. Система автоматически собирает данные из репозитория GitLab, рассчитывает метрики, опирающиеся на признанные исследовательские модели DORA и SPACE, и представляет результаты в форме ролевых дашбордов с парной визуализацией показателей «скорость ↔ качество».

Принципиальное отличие CherryGit от существующих аналогов состоит в том, что система спроектирована как **диагностический инструмент команд, а не как инструмент оценки или контроля сотрудников**. Эта установка реализована не декларативно, а архитектурно — через ролевую сегрегацию видимости данных на уровне REST API, парную визуализацию метрик и сознательное исключение показателей индивидуальной активности из управленческих дашбордов.

ГЛАВНЫЙ ТЕЗИС

CherryGit отвечает на вопрос «как помочь команде увидеть свой процесс и сделать его лучше?», а не на вопрос «как измерить работу каждого сотрудника?». Это смещение от измерительного инструмента к диагностическому определяет все ключевые проектные решения системы.

Краткие сведения о версии документа

Настоящая редакция концепции (версия 2.0) учитывает следующие изменения по сравнению с предыдущей редакцией:

- обновлено название продукта (с рабочего наименования на CherryGit);
- терминология приведена к академическому регистру: использован термин «*диагностика процессов разработки*» вместо ранее предлагавшегося термина «*инженерное здоровье*»;
- обновлена редакция модели DORA: учтён переход от исходных четырёх метрик к актуальной пяти-метричной модели, разделённой на группы Throughput и Instability; метрика MTTR заменена на Failed Deployment Recovery Time;
- добавлено явное методологическое обоснование на основе модели SPACE и закона Гудхарта;
- зафиксирована граница MVP в рамках выпускной квалификационной работы: интеграция только с GitLab, фокус на прикладной разработке информационной системы.

2. Цель системы и ожидаемый результат

2.1. Стратегическая цель

Стратегическая цель CherryGit — создать в IT-организации инфраструктуру непрерывного улучшения процессов разработки ПО на основе объективных данных. Система выступает технологическим фундаментом для перехода от субъективных оценок инженерной эффективности к подходу, основанному на воспроизводимых измерениях процесса разработки.

Цель выпускной квалификационной работы формулируется как разработка информационной системы для диагностики процессов разработки программного обеспечения, обеспечивающей автоматизированный сбор данных из репозитория GitLab, расчёт метрик процессов разработки на основе моделей DORA и SPACE, а также визуализацию и экспорт результатов.

ОЖИДАЕМЫЙ РЕЗУЛЬТАТ РАБОТЫ

Разработанная информационная система, обеспечивающая: подключение к произвольному инстансу GitLab (self-hosted или cloud); периодический инкрементальный сбор данных о коммитах, merge requests, ревью и релизах; расчёт ключевых метрик процессов разработки (Lead Time for Changes, Deployment Frequency, Change Failure Rate, Cycle Time MR с декомпозицией по фазам, MR Size, Bus Factor по модулям); веб-интерфейс с тремя ролевыми дашбордами, реализующий принципы парной визуализации показателей и прозрачности алгоритмов расчёта; ролевое разграничение доступа на уровне REST API. Результаты могут быть использованы командами разработки и техническими руководителями для выявления узких мест в процессах поставки ПО и принятия обоснованных решений по совершенствованию инженерных практик.

2.2. Операционные цели

Достижение стратегической цели декомпозируется на четыре операционные цели, каждая из которых направлена на свою аудиторию и решает свою группу задач.

Цель 1. Обеспечить команды объективной обратной связью по их процессу разработки

Команда должна иметь возможность регулярно и без существенных усилий получать объективную картину того, как устроен её процесс поставки ПО, где он тормозит, где деградирует качество, и какие изменения рабочих практик действительно улучшают результат. Без такой обратной связи команды вынуждены полагаться на интуицию и неформальные впечатления, что ограничивает их способность к самосовершенствованию.

Цель 2. Обеспечить тимлидов диагностическим инструментом состояния команды

Тимлид должен иметь возможность видеть состояние команды как системы: распределение нагрузки, бутылочные горлышки в код-ревью, риски концентрации знаний на отдельных участниках. При этом инструментарий должен явно разделять диагностику процесса (что показывает система) и оценку людей (что остаётся прерогативой непосредственного руководителя в рамках индивидуального диалога).

Цель 3. Обеспечить руководство объективными данными для решений об инженерной культуре

Руководитель отдела разработки должен иметь возможность сравнивать эффективность инженерных практик между командами, оценивать результаты инвестиций в улучшение процессов (внедрение CI/CD, реорганизация ревью, обучение команд) и принимать обоснованные решения о развитии инженерной культуры на уровне организации.

Цель 4. Защитить интересы и приватность сотрудников

Система должна архитектурно гарантировать, что сбор данных о работе разработчиков не превратится в инструмент микроменеджмента или дисциплинарного контроля. Это требование — не ограничение системы, а её ключевое отличие от существующих аналогов: только при соблюдении приватности команды готовы добровольно использовать платформу и доверять её данным.

3. Задачи системы

Для достижения сформулированных целей система решает четыре группы прикладных задач. Группы соответствуют разным аналитическим уровням и разным типам пользователей платформы.

3.1. Задачи уровня поставки ПО

Эта группа задач направлена на измерение скорости и надёжности процесса доставки изменений в продакшен — фундаментальной характеристики инженерной зрелости организации.

- Измерять время прохождения изменений от первого коммита по задаче до её попадания в продакшен (Lead Time for Changes), выявляя длительные участки и причины задержек.
- Измерять частоту релизов на уровне команды и репозитория (Deployment Frequency), оценивая способность организации доставлять ценность пользователю короткими итерациями.
- Измерять стабильность поставки через долю деплоев, потребовавших отката или экстренного исправления (Change Failure Rate), не позволяя оптимизировать скорость в ущерб качеству.
- Измерять время восстановления после неудачных деплоев (Failed Deployment Recovery Time), оценивая устойчивость инженерной системы к отказам.
- Поддерживать парную визуализацию метрик скорости и качества, делая невозможным просмотр одного без другого.

3.2. Задачи уровня код-ревью

Эта группа задач направлена на диагностику процесса коллегиальной проверки кода, который в большинстве команд является основным источником задержек и наиболее подверженной деградации частью процесса.

- Декомпозировать время жизни запроса на слияние на фазы: время до первого ревью, время в ревью, время от апрува до мержа — для точной локализации задержек.
- Анализировать размер запросов на слияние, выявляя случаи, когда merge request слишком крупен для качественного ревью.
- Измерять покрытие ревью и выявлять концентрацию ревью на ограниченном круге участников, что является риском устойчивости команды.
- Отслеживать долю возвратов и откатов (reopen, revert) как индикатор качества прохождения ревью.

3.3. Задачи диагностики команды

Эта группа задач направлена на выявление организационных рисков, которые не видны в обычной операционной деятельности, но накапливаются и проявляются в виде перегрузки участников, потери знаний и снижения устойчивости команды.

- Анализировать распределение нагрузки внутри команды для предотвращения концентрации работы на отдельных участниках.
- Оценивать bus factor по областям кода, выявляя модули, которые поддерживает только один разработчик.
- Анализировать паттерны обмена знаниями через граф «кто кого ревьюит», обнаруживая скрытые силосы внутри команды.

3.4. Задачи защиты прав пользователей

Эта группа задач является сквозной и реализуется на архитектурном уровне всех остальных модулей системы.

- Реализовать ролевую сегрегацию видимости данных на уровне REST API, делая технически невозможным доступ руководителя к индивидуальным метрикам участников команды без их согласия.
- Обеспечить прозрачность алгоритмов расчёта каждой метрики через раскрывающиеся блоки с формулами в пользовательском интерфейсе.
- Обеспечить минимизацию обработки персональных данных в соответствии с принципами ФЗ-152 за счёт явного разделения зон ответственности.
- Поддерживать развёртывание в инфраструктуре заказчика через docker-compose для организаций с повышенными требованиями к локализации данных.

4. Целевая аудитория

CherryGit обслуживает четыре аудитории, каждая из которых получает собственное ценностное предложение и работает со своим набором функций системы. Принципиально, что аудитории получают разные срезы одних и тех же данных, а не разные данные, причём правила сегрегации заложены архитектурно.

4.1. Разработчик

Разработчик: инструмент саморефлексии

БОЛЕВАЯ ТОЧКА

Отсутствие систематизированной обратной связи о собственной работе. Невозможность отследить динамику личных показателей и сравнить их с командным baseline. Затруднения с планированием профессионального развития.

ЦЕННОСТНОЕ ПРЕДЛОЖЕНИЕ

Приватный кабинет самоанализа: разработчик видит свою динамику работы за выбранный период, паттерны взаимодействия с командой, особенности своих запросов на слияние. Эти данные принадлежат ему, не передаются автоматически руководству и используются для самостоятельной рефлексии.

4.2. Тимлид команды

Тимлид: диагностический инструмент

БОЛЕВАЯ ТОЧКА

Сложность объективной оценки состояния процессов команды без погружения в каждый merge request. Запоздалое обнаружение признаков перегрузки участников, накопления технического долга, деградации качества ревью.

ЦЕННОСТНОЕ ПРЕДЛОЖЕНИЕ

Дашборд диагностики команды, отображающий агрегированные метрики уровня команды, декомпозированное время жизни MR, граф обмена знаниями, распределение нагрузки. Дополнительно — система сигналов аномалий, которая обращает внимание тимлида на устойчивые отклонения от командного baseline без раскрытия конкретных индивидуальных значений.

4.3. Руководитель отдела разработки

Руководитель отдела: инструмент стратегической диагностики

БОЛЕВАЯ ТОЧКА

Отсутствие сравнимых данных между командами. Невозможность объективно оценить, дают ли инвестиции в инженерные практики (внедрение CI/CD, обучение, реорганизация процессов) ожидаемый результат. Опора на субъективные впечатления тимлидов при принятии стратегических решений.

ЦЕННОСТНОЕ ПРЕДЛОЖЕНИЕ

Кросс-командные дашборды метрик потока (DORA-показатели), позволяющие сравнивать команды по единой шкале и оценивать эволюцию показателей во времени. Технически отсутствует возможность «провалиться» на уровень отдельного человека, что предотвращает соблазн микроменеджмента и защищает приватность сотрудников.

4.4. Организация в целом

На уровне организации CherryGit выступает инструментом снижения организационных рисков и повышения зрелости инженерной культуры.

- Снижение риска потери знаний и устойчивости команды через раннее обнаружение перегрузки участников и неравномерности нагрузки.
- Снижение bus-factor-рисков через выявление модулей с единственным владельцем и стимулирование обмена знаниями.
- Объективизация инвестиционных решений в инженерные практики и инструменты разработки через измеримый эффект на метриках DORA.
- Соответствие принципам защиты персональных данных сотрудников за счёт встроенной приватности и минимизации обработки.

5. Теоретическая основа: DORA и SPACE

CherryGit опирается на две взаимодополняющие исследовательские модели измерения процессов разработки ПО: DORA и SPACE. Обе модели имеют признанную научную базу и широко применяются в индустрии. В настоящем разделе фиксируется текущая редакция каждой модели и обосновывается её применение в проектируемой системе.

5.1. Модель DORA (текущая редакция, 5 метрик)

DORA (DevOps Research and Assessment) — исследовательская программа, изучающая возможности, определяющие производительность доставки ПО и операционную производительность ИТ-организаций. На основе многолетнего исследования тысяч команд DORA выделила набор метрик, демонстрирующих устойчивую корреляцию с производительностью команд разработки и благополучием их участников.

Важно отметить, что модель DORA эволюционировала. Изначально она содержала четыре метрики, известные как «the four keys». В текущей редакции метрики разделены на две группы и расширены до пяти показателей. Метрика Mean Time to Recovery (MTTR) была заменена на Failed Deployment Recovery Time для более точного отражения семантики измерения. Также введена пятая метрика Deployment Rework Rate.

Группа Throughput (пропускная способность)

Change Lead Time — время прохождения изменений

Время от момента, когда изменение зафиксировано в системе контроля версий, до момента, когда оно успешно работает в продакшене.

Deployment Frequency — частота деплоев

Количество успешных развёртываний в продакшен за единицу времени либо интервал между ними.

Failed Deployment Recovery Time — время восстановления

Время, необходимое для восстановления после неудачного деплоя, потребовавшего немедленного вмешательства. Метрика заменила прежний MTTR.

Группа Instability (нестабильность)

Change Fail Rate — доля неудачных изменений

Доля деплоев, потребовавших немедленного вмешательства после развёртывания (откат, хотфикс, fix forward, патч).

Deployment Rework Rate — доля переделок

Доля деплоев, которые были незапланированными и произошли как реакция на инцидент в продакшене.

КЛЮЧЕВОЙ ВЫВОД ИССЛЕДОВАНИЙ DORA

Скорость и стабильность не являются взаимоисключающими характеристиками: команды-лидеры демонстрируют высокие показатели по всем пяти метрикам одновременно, а отстающие команды — низкие показатели по всем метрикам. Это эмпирически опровергает распространённое представление об обмене скорости на качество.

5.2. Модель SPACE

SPACE — фреймворк измерения продуктивности разработчиков, предложенный в 2021 году исследователями Microsoft и GitHub. Основная идея фреймворка состоит в том, что продуктивность разработки нельзя свести к одному показателю, и для адекватной диагностики необходимо одновременно учитывать несколько граней. Название модели представляет собой акроним пяти измерений.

ИЗМЕРЕНИЕ		СОДЕРЖАНИЕ
S	Satisfaction and Well-Being Удовлетворённость и благополучие	Насколько разработчики довольны своей работой, инструментами и процессами. Преимущественно измеряется через опросы.
P	Performance Результативность работы	Качество и ценность создаваемого ПО: число багов, инциденты, удовлетворённость пользователей, бизнес-эффект.
A	Activity Активность	Объём измеряемых действий: коммиты, MR, ревью, деплои. Самые лёгкие для сбора, но самые опасные при использовании в отрыве от других граней.
C	Communication and Collaboration Коммуникация и сотрудничество	Эффективность взаимодействия: время ревью, bus factor, паттерны обмена знаниями.
E	Efficiency and Flow Эффективность и состояние потока	Насколько слаженно работа проходит путь от идеи до продакшена. Здесь располагаются метрики DORA.

Применительно к CherryGit модель SPACE используется как методологическая рамка, обеспечивающая многомерность диагностики: система намеренно не сводит все показатели к единому индексу, а представляет картину по нескольким независимым измерениям, не позволяя пользователю упустить ни одну из значимых граней.

5.3. Закон Гудхарта как методологический ограничитель

Закон Гудхарта формулируется следующим образом: «Когда мера становится целью, она перестаёт быть хорошей мерой». Применительно к измерению процессов разработки этот закон означает, что любая метрика, превращённая в KPI для оценки сотрудников, начнёт «надуваться» в ущерб реальной работе.

Классические примеры действия закона Гудхарта в IT-индустрии:

- введение счётчика коммитов на разработчика приводит к инфляции мелких коммитов и засорению истории репозитория;
- премирование за количество написанных строк кода стимулирует многословный код и наказывает за рефакторинг, удаляющий код;
- оценка по числу закрытых задач провоцирует дробление одной задачи на множество фиктивных подзадач.

МЕТОДОЛОГИЧЕСКОЕ СЛЕДСТВИЕ ДЛЯ CHERRYGIT

Закон Гудхарта учтён в архитектуре системы: индивидуальные метрики доступны только самому разработчику, командные метрики не персонализируются для руководства, парная визуализация скорости и качества встроена в интерфейс как ограничение, не позволяющее оптимизировать одну ось в ущерб другой. Таким образом, CherryGit реализует принцип «метрики измеряют процесс, а не людей» не декларативно, а архитектурно.

6. Принципы системы метрик

Прежде чем перейти к перечню метрик, необходимо зафиксировать четыре базовых принципа, на которых строится вся методология. Эти принципы определяют, какие метрики попадают в систему, как они визуализируются и как пользователю предлагается их интерпретировать.

ПРИНЦИП 1

Метрики измеряют процесс, а не людей

Любая метрика, попадающая в систему, рассматривается как характеристика процесса, а не как оценка человека. Этот принцип основан на законе Гудхарта: метрика, ставшая целью оценки сотрудника, перестаёт быть достоверным показателем. Принцип реализуется не на уровне рекомендаций, а на уровне архитектуры: индивидуальные показатели недоступны руководству.

ПРИНЦИП 2

Метрики результата важнее метрик активности

Система измеряет outcome (результат, который получает пользователь ПО), а не input (количество усилий, потраченных командой). Объём коммитов и строк кода — это input. Время от идеи до её работы у пользователя и стабильность этой работы — outcome. Управленческие решения, опирающиеся на input-метрики, систематически порождают неверные стимулы.

ПРИНЦИП 3

Парные метрики против односторонней оптимизации

Каждая метрика скорости в системе визуализируется только в паре с метрикой качества. Невозможно открыть дашборд частоты деплоев, не увидев рядом долю отказов; невозможно увидеть время мержа MR без доли возвратов и откатов. Это не рекомендация, а архитектурное ограничение интерфейса, защищающее от частой патологии — оптимизации одной оси за счёт другой.

ПРИНЦИП 4

Прозрачность алгоритмов расчёта

Формула расчёта каждой метрики и используемые источники данных доступны прямо в пользовательском интерфейсе через раскрывающийся блок при каждом графике. Это снимает фундаментальный барьер доверия команд к системе: разработчик, понимающий, как считается метрика, готов её обсуждать и улучшать; разработчик, не понимающий, будет либо игнорировать её, либо манипулировать ею.

7. Метрики системы

Метрики CherryGit распределены по трём аналитическим уровням, каждый из которых решает свою группу задач. Отдельно зафиксирован маппинг метрик на грани модели SPACE и перечень намеренно исключённых метрик.

7.1. Метрики потока поставки (DORA)

Ядро системы — метрики, выделенные исследовательской группой DORA на основе многолетнего изучения тысяч команд разработки. В CherryGit реализуются четыре из пяти метрик текущей редакции DORA; метрика Deployment Rework Rate отнесена в перспективные направления развития, так как требует интеграции с системой инцидент-менеджмента.

Lead Time for Changes — время прохождения изменений

Время от первого коммита по задаче до момента, когда изменение оказалось в продакшене. Главная метрика состояния процесса доставки. Длинный lead time почти всегда означает наличие узкого места — либо в код-ревью, либо в тестировании, либо в релизном цикле.

```
LT = timestamp(deploy_to_prod) - timestamp(first_commit_in_branch)
```

Отображается медиана и 90-й перцентиль за период.

Какую задачу решает: позволяет тимлиду и руководителю локализовать узкое место процесса. Команда с медианой 3 дня и команда с медианой 3 недели работают в принципиально разных режимах разработки, что требует разных управленческих решений.

Deployment Frequency — частота деплоев

Количество успешных деплоев в продакшен за единицу времени. Команда, релизнящаяся ежедневно, имеет принципиально более короткую петлю обратной связи и меньший риск на один релиз, чем команда с релизом раз в две недели.

```
DF = count(successful_deploys) / period
```

Категории по DORA: Elite (несколько раз в день), High (от раза в день до раза в неделю), Medium (от раза в неделю до раза в месяц), Low (реже).

Какую задачу решает: даёт руководству объективную оценку зрелости инженерных практик и измеримую цель для их улучшения.

Change Failure Rate — доля неудачных изменений

Процент деплоев, после которых потребовалось экстренное исправление, откат или хотфикс. Парная метрика к Deployment Frequency: без неё стремление к высокой частоте релизов превращается в гонку за скоростью в ущерб надёжности.

$$CFR = \text{count}(\text{deploys_with_hotfix_or_revert}) / \text{count}(\text{all_deploys}) \times 100\%$$

Какую задачу решает: защищает команду и руководство от ошибочного вывода «релиз часто = работаем хорошо». Команда с $DF=10/\text{день}$ и $CFR=40\%$ объективно работает хуже команды с $DF=2/\text{день}$ и $CFR=5\%$.

Failed Deployment Recovery Time — время восстановления после неудачного деплоя

Время от обнаружения неудачного деплоя до полного восстановления работоспособности сервиса. Характеризует устойчивость инженерной системы к отказам и зрелость процессов реагирования. Заменяла прежнюю метрику MTTR в актуальной редакции DORA.

$$FDRT = \text{average}(\text{timestamp}(\text{deploy_restored}) - \text{timestamp}(\text{failed_deploy_detected}))$$

Какую задачу решает: отвечает на вопрос «как быстро мы справляемся, когда деплой ломается». Низкое значение в сочетании с умеренным CFR — признак зрелой команды. Высокое значение — сигнал о проблемах с мониторингом, диагностикой или владением кодом.

7.2. Метрики процесса код-ревью

Этот блок метрик целиком извлекается из данных GitLab и не требует интеграции с CI/CD. Код-ревью — наиболее частая зона деградации процесса разработки, и качественная его диагностика даёт наибольшую отдачу при наименьшем объёме инженерных усилий.

Cycle Time MR с декомпозицией по фазам

Время жизни запроса на слияние, разделённое на три фазы:

- **Time to first review** — от открытия MR до первого комментария ревьюера. Если медиана превышает рабочий день, в команде нет культуры приоритета чужого ревью над своим кодом.
- **Time in review** — от первого ревью до апрува. Длинная фаза означает либо плохо подготовленный MR, либо разногласия в команде.
- **Time to merge after approval** — от апрува до мержа. Заметная фаза указывает на проблемы CI или ручные процессы.

Какую задачу решает: локализует узкое место процесса ревью. Без декомпозиции тимлид видит только общее «MR висят 3 дня», но не знает, на каком этапе работать с проблемой.

MR Size — размер запросов на слияние

Распределение размеров MR в команде по числу изменённых строк и затронутых файлов. Эмпирически известно: MR размером более 400 строк ревьюются формально, фактически штампуются апрувом без вдумчивой проверки.

Распределение MR по бакетам: ≤50, 51–200, 201–400, 401–800, >800 строк.

Какую задачу решает: сигнализирует, что в команде ревью существует формально, но не выполняет своей функции. Не для наказания авторов крупных MR, а для запуска разговора о практиках дробления задач.

Review Coverage — покрытие ревью

Доля MR, прошедших содержательное ревью (с комментариями, итерациями, более чем одним участником), и распределение нагрузки ревью между участниками команды.

```
RC = count(MR_with_real_review) / count(all_MR)
```

Дополнительно: концентрация ревью (доля участника-топ-1 в общем числе ревью).

Какую задачу решает: выявляет «вечных ревьюеров» как риск bus factor и культурный риск (если 70% ревью делает один человек, риск его перегрузки неизбежен). Помогает запускать практики ротации ревью.

Reopen / Revert Rate — доля возвратов и откатов

Процент MR, которые были переоткрыты после закрытия, или коммитов, которые были отменены через revert после мержа. Парная метрика к скорости ревью: высокая скорость мержа в сочетании с высокой долей откатов означает, что ревью не выполняет защитной функции.

Какую задачу решает: защищает от ложного оптимизма «у нас быстрое ревью». Если каждый десятый мерж приходится откатывать, скорость ревью не достижение, а проблема.

7.3. Метрики устойчивости команды

Эти метрики отвечают на вопросы организационной устойчивости: устойчивости к перегрузке участников, потере знаний, скрытым силосам. Они работают только на уровне команды и не должны экстраполироваться на индивидуальный уровень.

Workload Distribution — распределение нагрузки

Коэффициент Джини, рассчитанный по распределению активности (MR, ревью, коммиты) между участниками команды. Значение 0 — идеальная равномерность, значение 1 — вся работа на одном человеке.

$$G = (\sum_i \sum_j |x_i - x_j|) / (2n^2\bar{x})$$

где x — активность участника, n — число участников команды.

Какую задачу решает: предупреждает перегрузку и снижает риски, связанные с уходом ключевого участника. Не для выравнивания «всем поровну», а для обнаружения опасной концентрации.

Bus Factor по модулям

Для каждого модуля кода — число активных контрибьюторов за последние 90 дней. Модуль с единственным контрибьютором — это организационная проблема: уход одного человека парализует развитие модуля.

$$BF(module) = \text{count}(\text{distinct_authors with commits in last 90 days})$$

Какую задачу решает: делает видимыми скрытые организационные риски и обосновывает инвестиции во внутренние обмены знаниями, парное программирование, документацию.

Knowledge Sharing Index — индекс обмена знаниями

Граф, показывающий, кто кого ревьюит в команде. Здоровая команда имеет плотный граф со взаимными связями; команда со скрытыми силосами — несколько устойчивых пар без перекрёстных связей.

Какую задачу решает: обнаруживает невидимые иначе организационные паттерны — например, что разработчики А и В ревьюят только друг друга, изолируясь от остальной команды. Подобные структуры со временем превращаются во внутренние конфликты или потерю общего владения кодом.

7.4. Маппинг метрик на грани SPACE

Следующая таблица показывает, как метрики CherryGit покрывают грани модели SPACE. Маппинг подтверждает многомерность системы: ни одна грань не остаётся непокрытой, и ни одна не становится доминирующей. Грань Satisfaction намеренно остаётся за пределами автоматизированного сбора, поскольку требует регулярных опросов команды и не извлекается из данных Git-хостинга.

ГРАНЬ SPACE	МЕТРИКИ CHERRYGIT	СПОСОБ ПОКРЫТИЯ
S — Satisfaction	—	Вне автоматизированного сбора; требует опросов команды.
P — Performance	Change Failure Rate, Reopen / Revert Rate	Качество результата работы команды.
A — Activity	MR Size, Deployment Frequency	Объём измеряемых действий, всегда в паре с метриками качества.
C — Communication	Review Coverage, Knowledge Sharing Index, Bus Factor	Паттерны взаимодействия и обмена знаниями.
E — Efficiency / Flow	Lead Time, Cycle Time MR, Failed Deployment Recovery Time	Эффективность сквозного процесса от идеи до продакшена.

7.5. Намеренно исключённые метрики

Этот раздел не менее важен, чем перечень включённых метрик. Ряд показателей, традиционно собираемых аналогичными платформами, сознательно исключён из CherryGit, и эти исключения являются методологической позицией продукта.

ИСКЛЮЧЁННАЯ МЕТРИКА	ОБОСНОВАНИЕ ИСКЛЮЧЕНИЯ
Количество коммитов на разработчика	Метрика активности, не отражающая полезный результат. Один разработчик коммитит каждые 20 минут, другой — раз в день большим логичным куском. Оба продуктивны. Введение этого показателя порождает инфляцию мелких коммитов.
Lines of Code на разработчика	Хуже, чем бесполезно — вредно. Поощряет многословный код и наказывает за рефакторинг и удаление кода (одну из самых ценных активностей в зрелой кодовой базе).
Story Points в управленческих дашбордах	Story points — внутрикомандный инструмент планирования, не сравнимый между командами. При попадании в управленческую отчётность немедленно начинают инфлироваться.
Личные OKR и задачи в дашборде разработчика	Это не Git-метрика и не предметная область платформы. Их включение создаёт ложное впечатление, что система видит «полную картину» сотрудника, которую она в принципе не может видеть.
Индивидуальные дашборды для тимлида	Принципиальное архитектурное решение. Дашборд индивидуальных метрик доступен только самому разработчику. Тимлид видит командные агрегаты и сигналы аномалий без раскрытия конкретных индивидуальных значений.

8. Ролевая модель видимости данных

Один из ключевых дифференциаторов CherryGit — архитектурно реализованная сегрегация видимости данных. Принципиально, что это не настройка прав доступа, которую может изменить администратор, а встроенное в REST API свойство системы. Три уровня видимости описаны ниже.

УРОВЕНЬ / РОЛЬ	ЧТО ДОСТУПНО	ЧТО НЕДОСТУПНО
Разработчик	Полные индивидуальные метрики собственной работы. Командный baseline для сравнения. История своих метрик за всё время.	Индивидуальные метрики других участников команды.
Тимлид	Командные агрегаты по всем метрикам. Cycle Time MR с декомпозицией. Граф обмена знаниями (анонимизированный). Сигналы аномалий в формате «у участника N устойчивое отклонение от baseline».	Конкретные индивидуальные значения метрик участников команды без явного разрешения участника.
Руководитель отдела	Кросс-командные DORA-метрики. Сравнительная динамика команд. Агрегированные показатели по отделу в целом.	Любые индивидуальные данные. Любые данные глубже уровня команды.

АРХИТЕКТУРНАЯ ГАРАНТИЯ

Ролевая модель защищена не только на уровне UI, но и на уровне REST API. Запрос индивидуальных метрик другого пользователя со стороны тимлида или руководителя возвращает HTTP 403 на уровне бэкенда, а не маскируется на фронтенде. Это снимает целый класс атак на приватность данных и реализуется через middleware/dependency-механизм фреймворка.

9. Связь метрик с целями системы

Завершая описание системы метрик, важно зафиксировать, как каждая группа метрик работает на достижение целей, сформулированных в разделе 2 настоящего документа. Сводная таблица отвечает на вопрос «зачем вообще нужна каждая метрика» в терминах конкретных управленческих задач.

ЦЕЛЬ СИСТЕМЫ	ПОДДЕРЖИВАЮЩИЕ МЕТРИКИ	КАК МЕТРИКИ ПОМОГАЮТ
Обратная связь для команд по процессу	Cycle Time MR, MR Size, Review Coverage	Команда видит, где её процесс тормозит и какие практики ревью не работают, и может скорректировать их без внешнего давления.
Диагностика состояния команды для тимлида	Workload Distribution, Bus Factor, Knowledge Sharing Index, сигналы аномалий	Тимлид заранее видит риски перегрузки и потери знаний, может вовремя начать разговор с участником или перераспределить нагрузку.
Стратегические данные для руководства	Метрики DORA, кросс-командные сравнения, динамика метрик во времени	Руководитель видит измеримый эффект инвестиций в инженерные практики и принимает обоснованные решения о приоритетах развития культуры разработки.
Защита прав сотрудников	Архитектурная сегрегация видимости, прозрачность алгоритмов, развёртывание в инфраструктуре заказчика	Сотрудники доверяют системе и используют её добровольно; организация соответствует принципам ФЗ-152.

10. Граница MVP в рамках ВКР

Настоящий раздел фиксирует чёткую границу между общей концепцией CherryGit и объёмом, реализуемым в рамках выпускной квалификационной работы. Это разграничение позволяет защитить работу от ожиданий, не соответствующих реальному объёму прикладной разработки одного автора.

10.1. Объём MVP (обязательная часть)

КОМПОНЕНТ	РЕАЛИЗАЦИЯ В MVP
Источники данных	Интеграция только с GitLab (self-hosted и cloud) через REST API; авторизация по Personal Access Token.
Сбор данных	Периодический инкрементальный сбор: проекты, коммиты, merge requests, ревью, теги релизов.
Метрики потока	Lead Time for Changes, Deployment Frequency, Change Failure Rate (без Failed Deployment Recovery Time, требующего интеграции с системой мониторинга).
Метрики ревью	Cycle Time MR с декомпозицией на три фазы, MR Size.
Метрики команды	Bus Factor по модулям.
Интерфейс	Три ролевых дашборда: разработчик, тимлид, руководитель отдела. Парная визуализация скорости и качества. Раскрывающиеся блоки с формулами расчёта.
Ролевая модель	Реализация на уровне REST API через механизм Depends/Middleware; возврат HTTP 403 при попытке доступа вне роли.
Развёртывание	docker-compose с тремя контейнерами: backend, PostgreSQL, frontend.
Экспорт	Выгрузка отчётов по метрикам в формате CSV.

10.2. Направления развития (за пределами ВКР)

- Интеграция с GitHub и BitBucket через абстрактный коннектор источников данных.
- Интеграция с системами CI/CD (GitLab CI, Jenkins) для точного расчёта Failed Deployment Recovery Time и Deployment Rework Rate.
- Workload Distribution (коэффициент Джини), Knowledge Sharing Index, Reopen / Revert Rate.
- Система сигналов аномалий с настраиваемыми порогами и уведомлениями.
- Webhook-обновления данных в реальном времени взамен периодического сбора.
- Двухфакторная аутентификация, аудит-журнал доступа, шифрование БД на уровне записей.
- Интеграция с системами опросов команды для покрытия грани Satisfaction модели SPACE.

10.3. Допущения работы

- Деплои в продакшен определяются по тегам репозитория с настраиваемым паттерном; альтернативный механизм определения деплоев в MVP не реализуется.
- Хотфиксы и откаты определяются по меткам merge requests; альтернативный механизм через анализ commit message не реализуется.
- Команды моделируются как явные группы пользователей с привязкой к проектам GitLab; динамическое определение состава команды по активности не реализуется.

11. Заключение

CherryGit спроектирован как ответ на устойчивый разрыв на рынке аналитических инструментов для команд разработки. Существующие коммерческие платформы (Pluralsight Flow, Waydev, Gitalytics) в основном продают руководству инструмент контроля производительности сотрудников, что порождает сопротивление команд и этические вопросы. Open-source решения часто ограничиваются поверхностной статистикой коммитов, не давая управленческой ценности.

CherryGit занимает свободную нишу: **система диагностики процессов разработки, спроектированная в первую очередь для команд, а не для контроля над командами**. Этот сдвиг не является маркетинговым позиционированием — он реализован архитектурно через выбор метрик, парную визуализацию, ролевую сегрегацию данных и прозрачность алгоритмов.

КЛЮЧЕВЫЕ ТЕЗИСЫ КОНЦЕПЦИИ

- 1. Метрики измеряют процесс, а не людей. Закон Гудхарта учтён архитектурно: персональные метрики приватны, командные не персонализируются.*
- 2. Ядро метрик — модель DORA (5 метрик в актуальной редакции) и декомпозированный Cycle Time MR. Это исследовательски обоснованный набор, измеряющий результат, а не активность.*
- 3. Модель SPACE используется как методологическая рамка многомерной диагностики, не сводящей оценку к единому индексу.*
- 4. Парная визуализация скорости и качества встроена в UI как ограничение, защищающее от односторонней оптимизации.*
- 5. Ролевая сегрегация видимости реализована на уровне REST API и решает одновременно этическую, юридическую и маркетинговую задачу.*
- 6. Развёртывание в инфраструктуре заказчика и поддержка GitLab (включая self-hosted) делают систему пригодной для российского корпоративного рынка и организаций с повышенными требованиями к локализации данных.*

В совокупности эти решения делают CherryGit не «ещё одним инструментом аналитики Git», а инструментом диагностики процессов разработки — зеркалом, которое команды и организации используют для непрерывного улучшения своих инженерных практик.